

Regular Expressions

- String matching using a regular expression

Character(s)	Meaning
.	Matches any single character
^	Matches the null string at the start of the input string
\$	Matches the null string at the end of the input string
\m	Matches the null string at the start of a word
\M	Matches the null string at the end of a word
\k	Matches the nonalphanumeric character <i>k</i> (e.g., \. matches a literal . character)

Pattern	String	Result
an	"It is an Island."	Yes
lin	"It is an Island."	No
l.n	"It is an Island."	Yes
^an	"It is an Island."	No
^It i	"It is an Island."	Yes
nd\.\$	"It is an Island."	Yes
\mand	"It is an Island."	No
\mIsIa	"It is an Island."	Yes
\man\M	"It is an Island."	Yes

Regular Expressions

- String matching using a regular expression

<code>[chars]</code>	Matches any single character from <i>chars</i> . If the first character of <i>chars</i> is <code>^</code> , the pattern matches any single character not in the remainder of <i>chars</i> . A sequence of the form <i>a-b</i> in <i>chars</i> is treated as shorthand for all of the ASCII characters between <i>a</i> and <i>b</i> , inclusive. If the first character in <i>chars</i> (possibly following a <code>^</code>) is <code>]</code> , it is treated literally (as part of <i>chars</i> instead of a terminator). If a <code>-</code> appears first or last in <i>chars</i> , it is treated literally.
----------------------	--

Pattern	String	Result
<code>[hcr]at</code>	"A hat"	Yes
<code>[hcr]at</code>	"Brat"	Yes
<code>[hcr]at</code>	"fat"	No
<code>^[hcr]at</code>	"fat"	Yes
<code>^[hcr]at</code>	"Brat"	No
<code>[0-9]</code>	"5"	Yes
<code>[0-9]</code>	"a"	No
<code>[A-Z][0-9]</code>	"p7"	Yes
<code>[a-zA-Z0-9_]</code>	"B"	Yes
<code>^[a-zA-Z0-9_]</code>	"+"	Yes

Regular Expressions

*	Matches a sequence of zero or more matches of the preceding atom
+	Matches a sequence of one or more matches of the preceding atom
?	Matches either a null string or a match of the preceding atom
{ <i>m</i> }	Matches a sequence of exactly <i>m</i> matches of the preceding atom
{ <i>m</i> , }	Matches a sequence of <i>m</i> or more matches of the preceding atom
{ <i>m</i> , <i>n</i> }	Matches a sequence of <i>m</i> through <i>n</i> (inclusive) matches of the preceding atom

Pattern	String	Result
ba?b	"cbabde"	Yes
ba?b	"cbbde"	Yes
ba*b	"cbabde"	Yes
ba*b	"cbbde"	Yes
ba+b	"cbbde"	No
ba{2}b	"cbaabde"	Yes
ba{2,}b	"cbaaaabde"	Yes
ba{2,4}b	"cbaaaaabde"	No
[A-Z][0-9]+	"A78b"	Yes
[A-Z][0-9]+	"AB78b"	Yes

Regular Expressions

<code>(<i>regexp</i>)</code>	Matches anything that matches the regular expression <i>regexp</i> . Used for grouping and for identifying pieces of the matching substring.
<code><i>re1</i> <i>re2</i> ...</code>	Matches anything that matches any of the specified regular expressions

Pattern	String	Result
<code>(ab)+</code>	"cbabababde"	Yes
<code>(ab)+</code>	"cbbbbaade"	No
<code>(this that) car</code>	"this car"	Yes
<code>(this that) car</code>	"that car"	Yes
<code>(aa bb)+</code>	"aabbbaaaabb"	Yes
<code>(aa bb)+</code>	"ababa"	No
<code>^[+-]?[0-9]*(\.[0-9]+)?\$</code>	"+.580"	Yes

Regular Expressions

Class	Represents
<code>[:alpha:]</code>	Alphabetic letter
<code>[:alnum:]</code>	Alphanumeric letter or digit
<code>[:blank:]</code>	Space or tab character
<code>[:cntrl:]</code>	Control character
<code>[:digit:]</code>	Decimal digit
<code>[:graph:]</code>	Graphic character, a character with a visible representation such as a digit, alphabetic character, or punctuation
<code>[:lower:]</code>	Lowercase letter
<code>[:print:]</code>	Printable character, which includes all of the class <code>[:graph:]</code> but adds the space character
<code>[:punct:]</code>	Punctuation character
<code>[:space:]</code>	Whitespace character
<code>[:upper:]</code>	Uppercase letter
<code>[:xdigit:]</code>	Hexadecimal digit

Escape	Represents
<code>\d</code>	<code>[:digit:]</code>
<code>\D</code>	<code>^[[:digit:]]</code>
<code>\s</code>	<code>[:space:]</code>
<code>\S</code>	<code>^[[:space:]]</code>
<code>\w</code>	<code>[:alnum:]_</code>
<code>\W</code>	<code>^[[:alnum:]]_</code>

Regular Expressions

- Back referencing

Pattern	String	Result
<code>([a-z]+),([a-z]+)</code>	<code>"ab,cd"</code>	Yes
<code>([a-z]+),\1</code>	<code>"ab,cd"</code>	No
<code>([a-z]+),\1</code>	<code>"ab,ab"</code>	Yes
<code>(.)(.)\2\1</code>	<code>"abba"</code>	Yes
<code>(.)(.)\2\1</code>	<code>"abcd"</code>	No

- `\0` = Substring that matches the entire string
- `\1` = Substring that matches the subexpression in the first group
- `\2` = Substring that matches the subexpression in the second group
- ...

Regular Expressions

$([0-9]^+)*([a-z]^+)$

"Walk 10 km"

- $\backslash 0 = \text{"10 km"}$
- $\backslash 1 = \text{"10"}$
- $\backslash 2 = \text{"km"}$

Glob-Style Pattern Matching

- Glob-Style Pattern Matching

Character(s)	Meaning
*	Matches any sequence of zero or more characters
?	Matches any single character
[<i>chars</i>]	Matches any single character in <i>chars</i> . If <i>chars</i> contains a sequence of the form <i>a-b</i> , any character between <i>a</i> and <i>b</i> , inclusive, will match.
\ <i>x</i>	Matches the single character <i>x</i> . This provides a way to avoid special interpretation for any of the characters * ? [] \ in the pattern.

Pattern	String	Result
a*	"alphabet"	Yes
a*	"bat"	No
[ab]*	"brown"	Yes
a*	"Arizona"	No
*\?	"Wow!"	No
*\?	"What?"	Yes